HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA AN TOÀN THÔNG TIN

BÁO CÁO BÀI TẬP LỚN
HỌC PHẦN: LẬP TRÌNH VỚI PYTHON
MÃ HỌC PHẦN: INT13162

ĐỀ TÀI: NGHIÊN CỨU VÀ XÂY DỰNG MÔ HÌNH HỌC MÁY PHÁT HIỆN BẤT THƯỜNG DỰA TRÊN NHẬT KÝ HỆ THỐNG

Các sinh viên thực hiện:

B23DCAT156 Đỗ Trung Kiên

B23DCAT112 Nguyễn Huy Hoàng

B23DCAT172 Hà Duy Long

Tên nhóm: 03

Tên lớp: D23CTAT01-B

Giảng viên hướng dẫn: ThS. Vũ Minh Mạnh

HÀ NỘI 2026

PHÂN CÔNG NHIỆM VỤ NHÓM THỰC HIỆN

TT	Công việc / Nhiệm vụ	SV thực hiện	Thời hạn hoàn thành
1	Tìm hiểu tổng quan về phát hiện bất thường từ nhật ký và các phương pháp	Nguyễn Huy Hoàng	26/04/2026
2	Đề xuất mô hình phát hiện bất thường từ nhật ký hệ thống	Đỗ Trung Kiên	26/04/2026
3	Tiến hành thực nghiệm và rút ra đánh giá	Hà Duy Long	26/04/2026

NHÓM THỰC HIỆN TỰ ĐÁNH GIÁ

TT	SV thực hiện	Thái độ tham gia	Mức hoàn thành CV	Kỹ năng giao tiếp	Kỹ năng hợp tác	Kỹ năng lãnh đạo
1	Đỗ Trung Kiên	5	5	5	5	5
2	Nguyễn Huy Hoàng	5	5	5	5	5
3	Hà Duy Long	5	5	5	5	5

Ghi chú:

- Thái độ tham gia: Đánh giá điểm thái độ tham gia công việc chung của nhóm (từ 0: không tham gia, đến 5: chủ động, tích cực).
- Mức hoàn thành CV: Đánh giá điểm mức độ hoàn thành công việc được giao (từ 0: không hoàn thành, đến 5: hoàn thành xuất sắc).
- Kỹ năng giao tiếp: Đánh giá điểm khả năng tương tác, giao tiếp trong nhóm (từ 0: không hoặc giao tiếp rất yếu, đến 5: giao tiếp xuất sắc).
- Kỹ năng hợp tác: Đánh giá điểm khả năng hợp tác, hỗ trợ lẫn nhau, giải quyết mâu thuẫn, xung đột
- Kỹ năng lãnh đạo: Đánh giá điểm khả năng lãnh đạo (từ 0: không có khả năng lãnh đạo, đến 5: có khả năng lãnh đạo tốt, tổ chức và điều phối công việc trong nhóm hiệu quả).

MỤC LỤC

MỤC LỤC 3
DANH MỤC CÁC HÌNH VẼ 5
DANH MỤC CÁC BẢNG BIỂU 5
DANH MỤC CÁC TỪ VIẾT TẮT 6
MỞ ĐẦU 7
CHƯƠNG 1. TỔNG QUAN VỀ PHÁT HIỆN BẤT THƯỜNG TỪ NHẬT KÝ VÀ CÁC PHƯƠNG PHÁP PHÁT HIỆN 9
1.1 Tổng quan về phát hiện bất thường từ nhật ký hệ thống 9
1.2 Các phương pháp phát hiện bất thường từ nhật ký hệ thống 10
1.2.1 Phương pháp dựa trên luật và dấu hiệu 10
1.2.2 Phương pháp dựa trên thống kê và khai phá mẫu 10
1.2.3 Phương pháp học máy có giám sát 11
1.2.4 Phương pháp học máy không giám sát 12
1.2.5 Phương pháp học sâu 14
1.2.6 So sánh các phương pháp 14
1.3 Một số nghiên cứu liên quan về phát hiện bất thường từ nhật ký hệ thống 14
1.4 Kết chương 16
CHƯƠNG 2. ĐỀ XUẤT MÔ HÌNH PHÁT HIỆN BẤT THƯỜNG TỪ NHẬT KÝ HỆ THỐNG 17
2.1 Phát biểu bài toán 17
2.1.1 Input của bài toán 17
2.1.2 Output của bài toán 18
2.1.3 Mục tiêu của bài toán 18
2.2 Kiến trúc mô hình đề xuất 18
2.2.1 Cơ sở lựa chọn mô hình 18
2.2.2 Kiến trúc tổng thể của mô hình 19
2.3 Kết chương 22
CHƯƠNG 3. THỰC NGHIỆM VÀ ĐÁNH GIÁ 23
3.1 Mục tiêu thực nghiệm 23
3.2 Bộ dữ liệu và thiết lập thực nghiệm 23
3.3 Các độ đo đánh giá 24
3.4 Kết quả thực nghiệm trên HDFS 24
3.4.1 Kết quả của các mô hình có giám sát 24
3.4.2 Kết quả của các mô hình không giám sát 25

3.4.3 So sánh thời gian xử lý của các mô hình 26
3.4.4 Đánh giá khả năng mở rộng của các mô hình theo quy mô dữ liệu 27
3.5 Đánh giá và nhận xét 29
3.6 Kết chương 30
KẾT LUẬN 31
TÀI LIỆU THAM KHẢO 32

DANH MỤC CÁC HÌNH VẼ

Hình 1 – Ví dụ về Decision Tree.....12
Hình 2 - Quy trình xử lý log từ thu thập dữ liệu đến phát hiện bất thường 20
Hình 3 - Kiến trúc tổng thể của mô hình trong giai đoạn huấn luyện và giai đoạn phát hiện.....21
Hình 4 – So sánh thời gian xử lý của các mô hình.....27
Hình 5 – F1-score của các mô hình theo quy mô dữ liệu.....28
Hình 6 - Tổng thời gian xử lý của các mô hình theo quy mô dữ liệu 29

DANH MỤC CÁC BẢNG BIỂU

Bảng 1. Kết quả thực nghiệm của các mô hình có giám sát trên tập huấn luyện.....25
Bảng 2. Kết quả thực nghiệm của các mô hình có giám sát trên tập kiểm thử 25
Bảng 3. Kết quả thực nghiệm của các mô hình không giám sát trên tập huấn luyện.....26
Bảng 4. Kết quả thực nghiệm của các mô hình không giám sát trên tập kiểm thử.....26

DANH MỤC CÁC TỪ VIẾT TẮT

Từ viết tắt	Thuật ngữ tiếng Anh/Giải thích	Thuật ngữ tiếng Việt/Giải thích
PCA	Principal Component Analysis	Phân tích thành phần chính
HDFS	Hadoop Distributed File System	Hệ thống tệp phân tán Hadoop
LSTM	Long Short-Term Memory	Bộ nhớ ngắn-dài hạn
BERT	Bidirectional Encoder Representations from Transformers	Biểu diễn mã hóa hai chiều từ Transformer
BGL	Blue Gene/L	Bộ dữ liệu log Blue Gene/L
LR	Logistic Regression	Hồi quy logistic

MỞ ĐẦU

Nhật ký hệ thống (system log) là một trong những nguồn dữ liệu quan trọng phản ánh trạng thái hoạt động của các hệ thống phần mềm, đặc biệt là các hệ thống phân tán, hệ thống dịch vụ trực tuyến và hạ tầng công nghệ thông tin quy mô lớn [1][2]. Thông qua dữ liệu log, người quản trị có thể theo dõi tiến trình xử lý, phát hiện lỗi, nhận biết các hành vi bất thường và đánh giá tình trạng vận hành của hệ thống. Tuy nhiên, trong thực tế, số lượng log được sinh ra là rất lớn, đa dạng về định dạng và nội dung, khiến cho việc kiểm tra thủ công trở nên khó khăn, tốn nhiều thời gian và dễ bỏ sót các dấu hiệu bất thường [1][3].

So với nhu cầu giám sát và bảo trì hệ thống ngày càng cao, nhiều giải pháp phát hiện bất thường từ nhật ký hệ thống đã được đề xuất và triển khai sử dụng trên thực tế. Các giải pháp hiện nay có thể bao gồm: (1) các phương pháp phát hiện dựa trên luật, dấu hiệu hoặc mẫu bất thường đã biết và (2) các phương pháp phát hiện dựa trên học máy, học sâu nhằm học đặc trưng từ dữ liệu lịch sử. Nhìn chung, các phương pháp dựa trên luật hoặc dấu hiệu cho khả năng phát hiện nhanh và chính xác đối với các mẫu lỗi đã biết, song thường gặp khó khăn khi phải xử lý các bất thường mới hoặc các biến thể chưa từng xuất hiện. Ngoài ra, việc xây dựng và cập nhật tập luật, tập mẫu hoặc chữ ký thường đòi hỏi nhiều công sức của chuyên gia vận hành. Trong khi đó, các phương pháp dựa trên học máy và học sâu có ưu điểm là có thể tự động học từ dữ liệu, hỗ trợ phát hiện các bất thường mới và giảm phụ thuộc vào quá trình xây dựng đặc trưng thủ công. Tuy vậy, hiệu quả của các phương pháp này phụ thuộc mạnh vào chất lượng dữ liệu log, quy trình tiền xử lý, cách biểu diễn đặc trưng và lựa chọn mô hình phù hợp [2][4][5].

Một số nghiên cứu gần đây cho thấy các kỹ thuật phân tích log kết hợp học máy và học sâu có thể được sử dụng hiệu quả để xây dựng các mô hình phát hiện bất thường. Các hướng tiếp cận phổ biến bao gồm biểu diễn log dưới dạng vector đếm sự kiện, TF-IDF, khai phá quan hệ bất biến, phân tích thành phần chính, rừng cô lập, máy vector hỗ trợ, cũng như các mô hình học sâu xử lý chuỗi sự kiện log. Kết quả thực nghiệm trong nhiều nghiên cứu cho thấy việc kết hợp giữa bước chuẩn hóa log, trích xuất đặc trưng và mô hình học phù hợp có thể giúp nâng cao đáng kể hiệu quả phát hiện bất thường, đặc biệt trên các tập log hệ thống quy mô lớn [1][2][4].

Đề tài này tập trung nghiên cứu bài toán phát hiện bất thường từ nhật ký hệ thống và xây dựng một hệ thống phần mềm thử nghiệm hỗ trợ quá trình tiền xử lý log, trích xuất đặc trưng và áp dụng các mô hình học máy để phát hiện bất thường. Hệ thống được xây dựng theo hướng hỗ trợ thử nghiệm, đánh giá và so sánh nhiều phương pháp khác nhau trên cùng một tập dữ liệu, từ đó làm cơ sở lựa chọn mô hình phù hợp cho bài toán nghiên cứu. Cụ thể, các chức năng tiêu biểu của hệ thống bao gồm:

- Tiền xử lý và chuẩn hóa dữ liệu nhật ký hệ thống.
- Tách các bản ghi log thành các sự kiện có cấu trúc.
- Gom nhóm log theo phiên làm việc hoặc theo cửa sổ dữ liệu.

- Trích xuất đặc trưng từ chuỗi sự kiện log.
- Huấn luyện và áp dụng các mô hình học máy để phát hiện bất thường.
- Đánh giá kết quả theo các độ đo như Precision, Recall, F1-score và các chỉ tiêu liên quan.

Báo cáo bài tập lớn gồm 3 chương với nội dung chính như sau:

- Chương 1 nghiên cứu tổng quan về phát hiện bất thường từ nhật ký hệ thống và các phương pháp phát hiện, bao gồm các nội dung khái quát về dữ liệu log hệ thống, vai trò của log trong giám sát và chẩn đoán lỗi, bài toán phát hiện bất thường, các phương pháp phát hiện dựa trên luật, dấu hiệu, học máy và một số nghiên cứu liên quan.
- Chương 2 thực hiện việc phân tích, thiết kế và xây dựng mô hình phát hiện bất thường từ nhật ký hệ thống, trong đó bao gồm quy trình tiền xử lý log, biểu diễn dữ liệu, trích xuất đặc trưng, lựa chọn mô hình và kiến trúc tổng thể của hệ thống thử nghiệm.
- Chương 3 thực hiện việc thử nghiệm và đánh giá hệ thống phát hiện bất thường từ nhật ký hệ thống, trong đó bao gồm việc mô tả bộ dữ liệu, các độ đo đánh giá, tổ chức các kịch bản thực nghiệm, trình bày kết quả thu được và đưa ra các nhận xét, đánh giá đối với mô hình đã xây dựng.

CHƯƠNG 1. TỔNG QUAN VỀ PHÁT HIỆN BẤT THƯỜNG TỪ NHẬT KÝ VÀ CÁC PHƯƠNG PHÁP PHÁT HIỆN

Chương 1 trình bày cơ sở lý thuyết về phát hiện bất thường từ nhật ký hệ thống và các phương pháp phát hiện phổ biến. Nội dung chương tập trung làm rõ vai trò của dữ liệu log trong giám sát, chẩn đoán và bảo đảm an toàn cho hệ thống; đồng thời phân tích đặc điểm của bài toán phát hiện bất thường, các hướng tiếp cận dựa trên luật, dấu hiệu, thống kê, học máy và học sâu. Bên cạnh đó, chương cũng tổng hợp một số nghiên cứu liên quan, qua đó làm cơ sở cho việc đề xuất mô hình ở các chương tiếp theo.

1.1 Tổng quan về phát hiện bất thường từ nhật ký hệ thống

Nhật ký hệ thống là tập hợp các bản ghi được sinh ra trong quá trình thực thi của phần mềm, hệ điều hành, máy chủ hoặc các dịch vụ mạng. Mỗi bản ghi log thường chứa thời điểm phát sinh, thành phần sinh log, mức độ thông báo và nội dung thông điệp. Trong thực tế, log có thể phản ánh trạng thái vận hành bình thường, nhưng đồng thời cũng là nơi ghi nhận các lỗi phần mềm, lỗi cấu hình, tấn công, quá tải tài nguyên hoặc các hành vi sai lệch trong quá trình vận hành [1][5]. Chính vì vậy, log là nguồn dữ liệu rất quan trọng cho giám sát hệ thống và chẩn đoán sự cố.

Phát hiện bất thường từ log là quá trình xác định những mẫu log, chuỗi log hoặc phiên hoạt động có sự khác biệt đáng kể so với hành vi vận hành bình thường. Dưới góc nhìn bài toán học máy, ta có thể mô hình hóa như sau. Gọi tập log ban đầu là:

$$L = \{l_1, l_2, \dots, l_n\}$$

Sau bước tiền xử lý và chuẩn hóa, mỗi bản ghi log được ánh xạ thành một sự kiện có cấu trúc. Các sự kiện tiếp tục được nhóm theo phiên hoặc theo cửa sổ thời gian để tạo thành các chuỗi:

$$S = \{s_1, s_2, \dots, s_m\}$$

Từ mỗi chuỗi s_i, hệ thống trích xuất ra một vector đặc trưng x_i. Bài toán đặt ra là xây dựng một hàm:

$$f(x_i) \rightarrow y_i$$

trong đó $y_i \in \{0,1\}$, với $y_i = 0$ biểu diễn mẫu bình thường và $y_i = 1$ biểu diễn mẫu bất thường.

Từ phát biểu trên có thể thấy đầu vào của bài toán gồm dữ liệu log thô hoặc log đã được cấu trúc hóa, quy tắc chia chuỗi log theo phiên hoặc theo cửa sổ, tập nhãn trong trường hợp học có giám sát, cùng các tham số của mô hình. Đầu ra của hệ thống là nhãn dự đoán cho từng đơn vị log được xem xét, đồng thời có thể bao gồm các độ đo đánh giá như Precision, Recall, F1-score và thời gian xử lý.

So với các loại dữ liệu có cấu trúc truyền thống, log hệ thống có một số đặc điểm làm cho bài toán phát hiện bất thường trở nên khó khăn. Thứ nhất, log có tính không đồng nhất cao, được sinh ra từ nhiều thành phần khác nhau trong hệ thống. Thứ hai, log thường chứa

nhiều trường động như thời gian, biến đếm, địa chỉ, mã tiến trình hoặc mã phiên, làm cho số lượng mẫu câu bề ngoài trở nên rất lớn mặc dù về ngữ nghĩa có thể chỉ thuộc một số loại sự kiện hạn chế. Thứ ba, dữ liệu bất thường thường khan hiếm hơn rất nhiều so với dữ liệu bình thường, dẫn đến mất cân bằng lớp trong quá trình huấn luyện [1][2][4]. Chính vì vậy, một hệ thống phát hiện bất thường hiệu quả không chỉ phụ thuộc vào mô hình phân loại mà còn phụ thuộc chặt chẽ vào toàn bộ quy trình tiền xử lý và biểu diễn dữ liệu.

Có ba nguyên nhân chính khiến các hệ thống hiện đại cần phát hiện bất thường tự động từ log. Thứ nhất, hệ thống càng lớn thì hành vi tổng thể càng phức tạp, trong khi mỗi lập trình viên thường chỉ nắm rõ một phần của hệ thống. Điều đó khiến việc đọc log và suy luận thủ công trở nên khó khăn, dễ bỏ sót hoặc đánh giá sai. Thứ hai, log được sinh ra với tốc độ rất cao. Trong các hệ thống lớn, lượng log có thể lên tới hàng chục gigabyte mỗi giờ. Với tốc độ như vậy, ngay cả những công cụ tìm kiếm mạnh cũng chỉ giúp lọc bớt dữ liệu, chứ không thể thay thế hoàn toàn việc phân tích tự động. Thứ ba, các hệ thống lớn thường được thiết kế với cơ chế chịu lỗi, cơ chế sao chép, cơ chế thực thi song song hoặc suy đoán, khiến một số thông điệp có vẻ đáng ngờ thực ra lại hoàn toàn bình thường. Nếu chỉ dựa trên từ khóa cảnh báo thì số lượng cảnh báo giả sẽ rất cao, làm tăng công sức kiểm tra thủ công. Vì vậy, cần có các mô hình học được mẫu vận hành thực sự của hệ thống để đưa ra quyết định chính xác hơn.

1.2 Các phương pháp phát hiện bất thường từ nhật ký hệ thống

Các phương pháp phát hiện bất thường từ log có thể chia thành một số nhóm chính: phương pháp dựa trên luật hoặc tri thức chuyên gia, phương pháp dựa trên thống kê, phương pháp học máy truyền thống và phương pháp học sâu.

1.2.1 Phương pháp dựa trên luật và dấu hiệu

Đây là nhóm phương pháp xuất hiện sớm trong thực tế vận hành. Người quản trị hoặc chuyên gia xây dựng các luật thủ công như tìm kiếm từ khóa lỗi, nhận diện mẫu câu nguy hiểm hoặc kiểm tra sự xuất hiện của một chuỗi sự kiện nhất định. Ưu điểm của cách tiếp cận này là đơn giản, dễ áp dụng và hiệu quả tốt với các sự cố đã biết. Tuy nhiên, nhược điểm lớn là phụ thuộc mạnh vào kinh nghiệm của người xây dựng luật, khó mở rộng cho hệ thống lớn và khó phát hiện các bất thường mới chưa từng xuất hiện [1].

1.2.2 Phương pháp dựa trên thống kê và khai phá mẫu

Nhóm phương pháp này giả định rằng hành vi bình thường của hệ thống tuân theo một số quy luật thống kê hoặc các quan hệ lặp lại. Khi dữ liệu hiện tại sai khác đáng kể so với phân bố thống kê hoặc vi phạm các quan hệ thường gặp, hệ thống sẽ coi đó là bất thường. Các kỹ thuật tiêu biểu bao gồm phân tích thành phần chính, khai phá quan hệ bất biến và phân cụm.

PCA là một kỹ thuật giảm chiều, đồng thời có thể sử dụng sai số tái tạo hoặc phần dư ngoài không gian thành phần chính như một tín hiệu bất thường. Ưu điểm của PCA là đơn giản và phù hợp khi dữ liệu có tính tương quan cao. Tuy nhiên, khả năng mô hình hóa các quan hệ phi tuyến còn hạn chế [1]. Phương pháp khai phá quan hệ bất biến tìm các quan hệ

ổn định giữa số lần xuất hiện của các sự kiện log trong trạng thái bình thường; khi quan hệ này bị phá vỡ thì hệ thống ghi nhận bất thường. Phân cụm thì dựa trên giả định rằng các mẫu bình thường tạo thành những cụm rõ ràng, còn mẫu bất thường nằm tách biệt khỏi các cụm chính [1][7].

1.2.3 Phương pháp học máy có giám sát

Trong nhóm này, dữ liệu huấn luyện đã có nhãn bình thường hoặc bất thường. Một số mô hình thường được sử dụng gồm Logistic Regression, Decision Tree và Support Vector Machine.

1.2.3.1 Logistic Regression

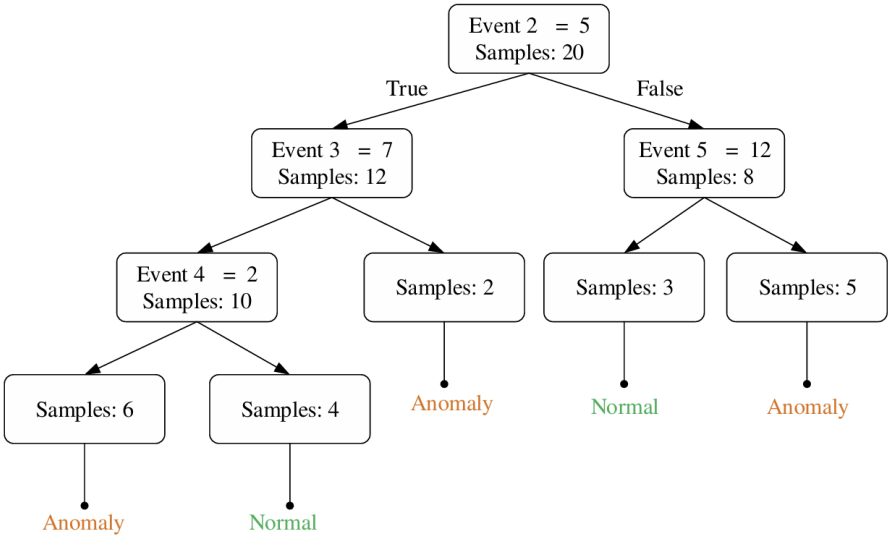
Logistic Regression là một mô hình phân loại dựa trên ước lượng xác suất. Với mỗi event count vector, mô hình tính xác suất nó thuộc lớp bất thường. Nếu xác suất vượt ngưỡng, chuỗi log sẽ bị gán là bất thường. Ưu điểm của phương pháp này là đơn giản, nhanh và dễ triển khai. Hạn chế là khả năng mô tả các quan hệ phi tuyến không mạnh bằng một số mô hình khác.

1.2.3.2 Decision Tree

Decision Tree là một dạng sơ đồ có cấu trúc phân nhánh, trong đó các nhánh được sử dụng để biểu diễn trạng thái dự đoán tương ứng với từng mẫu dữ liệu. Decision Tree được xây dựng theo phương pháp từ trên xuống, dựa trên dữ liệu huấn luyện. Mỗi nút của cây được hình thành từ thuộc tính tốt nhất tại thời điểm hiện tại, được lựa chọn dựa trên độ lợi thông tin (information gain) của thuộc tính đó.

Chẳng hạn, nút gốc trong Hình 1 cho thấy bộ dữ liệu gồm tổng cộng 20 mẫu. Khi thực hiện phân tách nút gốc, số lần xuất hiện của Sự kiện 2 được xem là thuộc tính tốt nhất. Do đó, toàn bộ 20 mẫu huấn luyện được chia thành hai tập con theo giá trị của thuộc tính này, trong đó một tập gồm 12 mẫu và tập còn lại gồm 8 mẫu.

Decision Tree lần đầu tiên được áp dụng cho bài toán chẩn đoán lỗi trong hệ thống nhật ký yêu cầu web ở nghiên cứu. Các event count vector cùng với nhãn tương ứng được sử dụng để xây dựng Decision Tree. Để xác định trạng thái của một mẫu mới, mẫu đó sẽ được duyệt qua Decision Tree dựa trên các điều kiện tại từng nút trên đường đi. Khi kết thúc quá trình duyệt, mẫu sẽ đi đến một nút lá, nút lá này biểu thị trạng thái của mẫu đó.



Hình 1 – Ví dụ về Decision Tree

1.2.3.3 Support Vector Machine

Support Vector Machine – SVM là một phương pháp học có giám sát được sử dụng trong bài toán phân loại. Trong SVM, một siêu phẳng (hyperplane) được xây dựng để phân tách các lớp mẫu khác nhau trong không gian nhiều chiều. Việc xác định siêu phẳng này là một bài toán tối ưu, với mục tiêu cực đại hóa khoảng cách giữa siêu phẳng và điểm dữ liệu gần nhất của các lớp khác nhau.

Tương tự như Logistic Regression và Decision Tree, các mẫu huấn luyện được biểu diễn dưới dạng các event count vector kèm theo nhãn tương ứng. Trong phát hiện bất thường bằng SVM, nếu một mẫu mới nằm phía trên siêu phẳng thì mẫu đó được xác định là bất thường, ngược lại, nó được xem là bình thường. Có hai loại SVM, gồm SVM tuyến tính (linear SVM) và SVM phi tuyến (non-linear SVM). Trong bài báo cáo này chỉ xem xét SVM tuyến tính, vì trong phần lớn các thí nghiệm, SVM tuyến tính cho kết quả tốt hơn SVM phi tuyến.

1.2.4 Phương pháp học máy không giám sát

Nhóm này không yêu cầu dữ liệu gán nhãn trong giai đoạn huấn luyện. Một số phương pháp tiêu biểu là Log Clustering, PCA, Invariant Mining.

1.2.4.1 Log Clustering

LogCluster là một phương pháp dựa trên phân cụm nhằm phát hiện sự cố trong hệ thống trực tuyến. Phương pháp này gồm hai giai đoạn huấn luyện: khởi tạo tập mẫu tham chiếu và học trực tuyến. Ở giai đoạn đầu, các chuỗi log được chuyển thành các event count vector, sau đó được hiệu chỉnh bằng IDF và chuẩn hóa. Tiếp theo, các vector này được phân cụm riêng thành nhóm bình thường và bất thường bằng phương pháp phân cụm phân cấp kết tụ, mỗi cụm sau đó được biểu diễn bởi một vector đại diện là trọng tâm của cụm.

Ở giai đoạn học trực tuyến, các vector mới được thêm lần lượt vào tập mẫu tham chiếu để cập nhật các cụm hiện có. Với mỗi vector đầu vào, hệ thống tính khoảng cách từ nó đến các vector đại diện, nếu khoảng cách nhỏ nhất nhỏ hơn ngưỡng cho trước, vector đó sẽ được gán vào cụm gần nhất và cập nhật lại vector đại diện, ngược lại sẽ tạo thành một cụm mới. Sau khi hoàn tất huấn luyện, LogCluster được dùng để phát hiện bất thường bằng cách so sánh chuỗi log mới với các vector đại diện trong cơ sở tri thức. Nếu khoảng cách nhỏ nhất vượt quá ngưỡng, chuỗi log được xem là bất thường, ngược lại, trạng thái của nó được xác định theo cụm gần nhất.

1.2.4.2 PCA

PCA là một phương pháp thống kê được sử dụng rộng rãi cho bài toán giảm chiều dữ liệu. Ý tưởng cơ bản của PCA là chiếu dữ liệu có số chiều lớn sang một hệ tọa độ mới gồm k thành phần chính (principal components), trong đó k nhỏ hơn số chiều ban đầu. PCA xác định các thành phần chính này bằng cách tìm ra những trục mang nhiều phương sai nhất của dữ liệu. Nhờ đó, dữ liệu sau khi giảm chiều vẫn bảo toàn được những đặc trưng quan trọng của dữ liệu gốc, chẳng hạn như mức độ tương đồng giữa các điểm dữ liệu.

Trong phương pháp này, mỗi chuỗi log được biểu diễn dưới dạng một event count vector. Sau đó, PCA được sử dụng để tìm ra các mẫu quan hệ giữa các chiều của các vector này. Kết quả là hai không gian con được tạo ra, gồm không gian bình thường S_n và không gian bất thường S_a. Không gian S_n được tạo thành từ k thành phần chính đầu tiên, còn S_a được tạo thành từ n − k thành phần còn lại, trong đó n là số chiều ban đầu. Tiếp theo, phép chiếu $y_a = (I - PP^T)y$ của event count vectors y lên không gian bất thường S_a được tính toán, với $P = [v_1, v_2, \ldots, v_k]$ là ma trận gồm k thành phần chính đầu tiên. Nếu độ dài của y_a lớn hơn một ngưỡng xác định, vector tương ứng sẽ được xem là bất thường. Cụ thể, một event count vector được xem là bất thường nếu:

$$SPE \equiv \| y_a \|^2 > Q_\alpha$$

trong đó sai số dự đoán bình phương (Squared Prediction Error – SPE) biểu thị độ dài của phép chiếu, còn Q_α là ngưỡng tương ứng với mức tin cậy $1 - \alpha$. Các nghiên cứu sẽ đặt $Q = 0{,}001$, và giá trị k được xác định tự động sao cho PCA giữ lại 95% phương sai của dữ liệu.

1.2.4.3 Invariant Mining

Invariant Mining dựa trên giả định rằng trong quá trình hệ thống vận hành, luôn tồn tại những quan hệ tuyến tính cố định giữa các sự kiện log, bất kể đầu vào hay khối lượng công việc thay đổi như thế nào. Phương pháp này lần đầu tiên được áp dụng cho phát hiện bất thường dựa trên log trong. Các log có cùng session id (ví dụ: block id trong HDFS) thường biểu diễn luồng thực thi chương trình của cùng một session.

Về trực giác, khai phá bất biến giúp phát hiện các quan hệ tuyến tính phản ánh hành vi thực thi bình thường của hệ thống. Chẳng hạn, trong điều kiện bình thường, một tệp sau khi được mở thì phải được đóng. Do đó, số log chứa cụm “open file” và “close file” thường xuất hiện theo cặp. Nếu trong một phiên thực thi, số lần xuất hiện của hai loại log này không

bằng nhau thì phiên đó có thể bị xem là bất thường vì vi phạm quan hệ tuyến tính vốn có. Quá trình khai phá bất biến gồm ba bước: trước hết, từ các chuỗi log tạo ra event count matrix, trong đó mỗi hàng là một event count vector. Tiếp theo, không gian bất biến được ước lượng bằng phân rã giá trị suy biến (singular value decomposition – SVD) để xác định số lượng bất biến cần khai phá. Sau đó, các bất biến được tìm bằng thuật toán vét cạn và được kiểm chứng thông qua mức độ hỗ trợ của chúng trong dữ liệu. Trong giai đoạn phát hiện bất thường, khi một chuỗi log mới xuất hiện, hệ thống sẽ kiểm tra xem chuỗi đó có thỏa mãn các bất biến đã khai phá hay không; nếu có ít nhất một bất biến bị vi phạm, chuỗi log đó sẽ được xem là bất thường.

1.2.5 Phương pháp học sâu

Sự phát triển của học sâu đã mở ra nhiều hướng tiếp cận mới cho bài toán phát hiện bất thường từ log. DeepLog mô hình hóa chuỗi log bằng mạng LSTM để học các mẫu tuần tự của hành vi bình thường; khi chuỗi sự kiện mới sai khác khỏi mẫu học được, hệ thống sẽ gán là bất thường. LogAnomaly mở rộng hướng tiếp cận này bằng cách kết hợp thông tin tuần tự và thông tin định lượng, đồng thời khai thác ngữ nghĩa của mẫu log thông qua biểu diễn vector. Các mô hình học sâu thường cho hiệu quả tốt hơn trong việc mô hình hóa ngữ cảnh chuỗi, song chi phí tính toán lớn hơn, yêu cầu dữ liệu huấn luyện nhiều hơn và khó diễn giải hơn so với các mô hình truyền thống.

1.2.6 So sánh các phương pháp

Để củng cố sự hiểu biết về cách tiếp cận phát hiện bất thường nêu trên, đồng thời để giúp lựa chọn phương pháp phù hợp hơn, có thể rút ra một số ưu điểm và hạn chế của các phương pháp.

Với các phương pháp có giám sát, nhãn là bắt buộc cho phát hiện bất thường. Decision Tree dễ diễn giải hơn hai phương pháp còn lại, vì các nhà phát triển có thể phát hiện bất thường với những giải thích có ý nghĩa (tức các điều kiện tại các nút của cây). Logistic Regression không thể giải quyết các bài toán không phân tách tuyến tính, trong khi SVM có thể giải quyết bằng các kernel. Tuy nhiên, các tham số của SVM khó tinh chỉnh (ví dụ tham số phạt), nên thường đòi hỏi nhiều công sức thủ công để thiết lập mô hình.

Các phương pháp không giám sát thực tế và có ý nghĩa hơn do thiếu nhãn. Phân cụm log sử dụng ý tưởng học trực tuyến, vì vậy phù hợp để xử lý khối lượng log lớn. Khai phá bất biến không chỉ có thể phát hiện bất thường với độ chính xác cao, mà còn có thể cung cấp diễn giải có ý nghĩa và trực quan cho mỗi bất thường được phát hiện. Tuy nhiên, quá trình khai phá bất biến tốn thời gian. PCA không dễ hiểu và nhạy với dữ liệu. Vì vậy, độ chính xác phát hiện bất thường của nó thay đổi giữa các bộ dữ liệu khác nhau.

1.3 Một số nghiên cứu liên quan về phát hiện bất thường từ nhật ký hệ thống

Trong những năm gần đây, nghiên cứu về phát hiện bất thường từ nhật ký hệ thống có xu hướng chuyển từ các phương pháp dựa trên đặc trưng đếm sự kiện và mô hình học máy truyền thống sang các phương pháp khai thác ngữ nghĩa sâu của thông điệp log. Một hướng đáng chú ý là giảm sự phụ thuộc vào bước log parsing và học trực tiếp biểu diễn của log

bằng các mô hình ngôn ngữ. Chẳng hạn, LAnoBERT là một phương pháp không cần parser, sử dụng masked language modeling trên nền BERT để phát hiện bất thường theo hướng không giám sát. Kết quả thực nghiệm trên ba bộ dữ liệu HDFS, BGL và Thunderbird cho thấy phương pháp này đạt hiệu năng cao hơn các mốc so sánh không giám sát và tiệm cận các mô hình có giám sát. Cùng hướng đó, CLDTLog kết hợp BERT với contrastive learning và cơ chế hai hàm mục tiêu, tác giả báo cáo F1-score đạt 0.9971 trên HDFS và 0.9999 trên BGL, và ngay cả khi chỉ sử dụng 1% dữ liệu huấn luyện trên BGL, mô hình vẫn đạt F1-score 0.9993. Những kết quả này cho thấy khai thác ngữ nghĩa của log là một hướng tiếp cận rất hiệu quả đối với bài toán phát hiện bất thường hiện nay.

Bên cạnh việc nâng cao chất lượng biểu diễn log, một số nghiên cứu gần đây tập trung vào bài toán phát hiện bất thường xuyên hệ thống, tức là tận dụng tri thức từ hệ thống nguồn để hỗ trợ phát hiện trên hệ thống đích có rất ít dữ liệu gán nhãn. Tiêu biểu cho hướng này là MetaLog được công bố tại ICSE 2024. Nghiên cứu này sử dụng biểu diễn ngữ nghĩa dùng chung kết hợp với meta-learning để tăng khả năng khái quát hóa giữa các hệ thống khác nhau. Theo kết quả công bố, mô hình đạt trên 80% F1-score khi chỉ sử dụng 1% log có nhãn của hệ thống đích, đồng thời vượt khoảng 20% so với các phương pháp phát hiện xuyên hệ thống dựa trên transfer learning trong cùng điều kiện thử nghiệm. Kết quả đó cho thấy khả năng thích nghi miền và khai thác tri thức liên hệ giữa các hệ thống đang trở thành một xu hướng nghiên cứu quan trọng trong lĩnh vực này.

Ngoài ra, một hướng nghiên cứu khác là tự động hóa quy trình phát hiện bất thường, nhằm giảm sự phụ thuộc vào chuyên gia khi triển khai trên tập log mới. Nghiên cứu LogCraft tại ASE 2024 đề xuất một khung AutoML đầu-cuối cho phát hiện bất thường không giám sát, tự động hóa các bước tạo đặc trưng, lựa chọn mô hình và phát hiện dị thường. Kết quả đánh giá trên năm bộ dữ liệu công khai cho thấy LogCraft đạt F1-score trung bình 0.830, cao hơn mức 0.778 của phương pháp không giám sát tốt thứ hai trong đối sánh của tác giả. Điều này cho thấy ngoài việc cải tiến mô hình phát hiện, các nghiên cứu gần đây còn hướng mạnh tới khả năng triển khai thực tế và giảm chi phí cấu hình hệ thống.

Ở góc nhìn đánh giá thực nghiệm quy mô lớn, nghiên cứu tổng hợp của Ali và cộng sự năm 2025 cũng đưa ra một nhận định đáng chú ý. Khi so sánh chín kỹ thuật học máy và học sâu trên nhiều bộ dữ liệu benchmark, nhóm tác giả cho thấy các phương pháp học có giám sát truyền thống và học sâu thường cho kết quả rất gần nhau về độ chính xác và thời gian dự đoán trên phần lớn bộ dữ liệu, đồng thời các mô hình truyền thống còn ít nhạy hơn với việc tinh chỉnh siêu tham số. Từ đó, nghiên cứu này khuyến nghị rằng trong nhiều trường hợp thực tế, các mô hình truyền thống, đặc biệt là Random Forest, vẫn là lựa chọn rất đáng cân nhắc trước khi chuyển sang các mô hình sâu phức tạp hơn. Nhận định này có ý nghĩa quan trọng vì nó cho thấy bài toán phát hiện bất thường từ log không chỉ là cuộc đua về mô hình mới, mà còn là bài toán lựa chọn giải pháp phù hợp với dữ liệu, chi phí tính toán và bối cảnh triển khai cụ thể.

Từ các nghiên cứu nêu trên có thể thấy rằng nghiên cứu hiện nay đang phát triển theo ba hướng chính: nâng cao chất lượng biểu diễn ngữ nghĩa của log, tăng khả năng thích nghi khi dữ liệu nhãn hạn chế hoặc thay đổi giữa các hệ thống, và tự động hóa quy trình phát hiện

bất thường. Tuy nhiên, các kết quả gần đây cũng đồng thời cho thấy chưa có một phương pháp nào luôn tối ưu trong mọi bối cảnh. Hiệu quả cuối cùng vẫn phụ thuộc nhiều vào đặc điểm của tập log, chất lượng tiền xử lý, cách biểu diễn đặc trưng và tiêu chí đánh giá thực nghiệm.

1.4 Kết chương

Chương 1 đã trình bày tổng quan về bài toán phát hiện bất thường từ nhật ký hệ thống, nhấn mạnh vai trò quan trọng của dữ liệu log trong giám sát hoạt động, phát hiện lỗi và hỗ trợ chẩn đoán sự cố trong các hệ thống công nghệ thông tin. Bên cạnh đó, chương đã phân tích các hướng tiếp cận tiêu biểu trong phát hiện bất thường, bao gồm phương pháp dựa trên luật, dấu hiệu, thống kê, học máy và học sâu, đồng thời chỉ ra ưu điểm, hạn chế và phạm vi áp dụng của từng phương pháp. Ngoài ra, việc tổng hợp một số nghiên cứu liên quan cũng cho thấy xu hướng ứng dụng mạnh mẽ các kỹ thuật học máy trong phân tích log hiện nay.

CHƯƠNG 2. ĐỀ XUẤT MÔ HÌNH PHÁT HIỆN BẤT THƯỜNG TỪ NHẬT KÝ HỆ THỐNG

Chương 2 trình bày mô hình phát hiện bất thường từ nhật ký hệ thống được đề xuất. Nội dung chương tập trung vào việc phát biểu bài toán, xác định dữ liệu đầu vào, đầu ra và mục tiêu cần đạt được, đồng thời mô tả kiến trúc tổng thể của mô hình, các bước phân tích cú pháp log, gom nhóm theo session window, trích xuất đặc trưng và huấn luyện mô hình. Ngoài ra, chương cũng trình bày quy trình thực hiện tổng quát nhằm làm rõ cách triển khai và kiểm thử mô hình trên dữ liệu thực tế.

2.1 Phát biểu bài toán

Bài toán đặt ra trong báo cáo này là xây dựng một mô hình có khả năng tự động phát hiện bất thường từ nhật ký hệ thống HDFS trên cơ sở bộ dữ liệu HDFS.npz. Về bản chất, mô hình cần học được quy luật phân biệt giữa các phiên thực thi bình thường và các phiên thực thi bất thường, từ đó đưa ra dự đoán cho các dữ liệu log chưa thấy trước đó.

2.1.1 Input của bài toán

Trong ngữ cảnh của HDFS, dữ liệu ban đầu là các dòng log được sinh ra trong quá trình vận hành hệ thống tệp phân tán. Mỗi dòng log chứa thông tin thời gian chạy và nội dung sự kiện, trong đó HDFS có trường block_id cho phép gom các dòng log liên quan đến cùng một khối dữ liệu thành một phiên thực thi (session).

Xét theo quy trình xử lý, đầu vào của bài toán có thể được mô tả ở hai mức như sau:

a) Input ở mức log thô

Đầu vào ban đầu là tập log HDFS:

$$L = \{l_1, l_2, \ldots, l_n\}$$

trong đó mỗi dòng log l_i có thể gồm các thông tin như:

- thời gian xuất hiện,
- nội dung thông điệp log,
- định danh khối dữ liệu (block_id),
- các tham số động khác của hệ thống.

Ở mức này, dữ liệu vẫn là dữ liệu văn bản bán cấu trúc, chưa thể đưa trực tiếp vào mô hình học máy.

b) Input ở mức dữ liệu sau tiền xử lý

Sau khi log được phân tích cú pháp, gom nhóm theo session window dựa trên block_id, và trích xuất đặc trưng, dữ liệu được biểu diễn thành ma trận đặc trưng

$$X \in R^{N \times m}.$$

trong đó:

- N là số phiên log (số session),
- m là số loại sự kiện log sau khi chuẩn hóa,
- mỗi hàng $x_i \in \mathbb{R}^m$ là một event count vector, biểu diễn số lần xuất hiện của từng sự kiện trong phiên thứ i.

Tương ứng với ma trận đặc trưng là vector nhãn $y = \{y_1, y_2, \ldots, y_N\}, y_i \in \{0,1\}$

trong đó :

- $y_i = 0$: phiên log bình thường,
- $y_i = 1$: phiên log bất thường.

Trong phạm vi thực nghiệm với HDFS.npz, có thể xem input trực tiếp của mô hình là cặp dữ liệu (X, y), tức ma trận đặc trưng và vector nhãn đã được chuẩn bị sẵn sau bước tiền xử lý.

2.1.2 Output của bài toán

Đầu ra của bài toán là kết quả phân loại cho mỗi phiên log đầu vào. Cụ thể, với một phiên log mới có vector đặc trưng x, mô hình cần đưa ra một hàm dự đoán:

$$f(x) \rightarrow \hat{y}, \hat{y} \in \{0,1\}$$

trong đó:

- $\hat{y} = 0$: phiên được dự đoán là bình thường,
- $\hat{y} = 1$: phiên được dự đoán là bất thường.

Như vậy, output cuối cùng của hệ thống là:

- Nhãn dự đoán cho từng phiên log,
- Trong một số mô hình, có thể kèm theo điểm số bất thường hoặc xác suất dự đoán, dùng để đánh giá mức độ bất thường của phiên đó.

2.1.3 Mục tiêu của bài toán

Từ input là các phiên log HDFS đã được biểu diễn dưới dạng vector đặc trưng, mục tiêu của mô hình là học được một bộ phân loại sao cho:

- phân biệt chính xác giữa các phiên bình thường và bất thường,
- giảm số lượng cảnh báo giả,
- đồng thời phát hiện được tối đa các phiên bất thường thực sự

2.2 Kiến trúc mô hình đề xuất

2.2.1 Cơ sở lựa chọn mô hình

Trong bài toán phát hiện bất thường từ nhật ký hệ thống HDFS, việc lựa chọn mô hình học máy cần bảo đảm đồng thời hai yêu cầu: hiệu quả phát hiện và sự phù hợp với đặc trưng dữ liệu. Sau khi dữ liệu được biểu diễn dưới dạng event count matrix, có thể thấy các phiên

log bình thường và bất thường có sự khác biệt tương đối rõ trên không gian đặc trưng, tạo điều kiện thuận lợi cho các mô hình phân loại học được ranh giới quyết định.

Trên cơ sở đó, báo cáo xem xét ba mô hình có giám sát gồm Logistic Regression, Decision Tree và SVM. Đây là các phương pháp phân loại điển hình, có khả năng làm việc tốt với dữ liệu dạng vector đặc trưng. Kết quả thực nghiệm cho thấy các mô hình này đều có khả năng phân biệt hai lớp dữ liệu trên bộ dữ liệu HDFS, chứng tỏ cách tổ chức log theo phiên và biểu diễn bằng số lần xuất hiện của sự kiện là phù hợp.

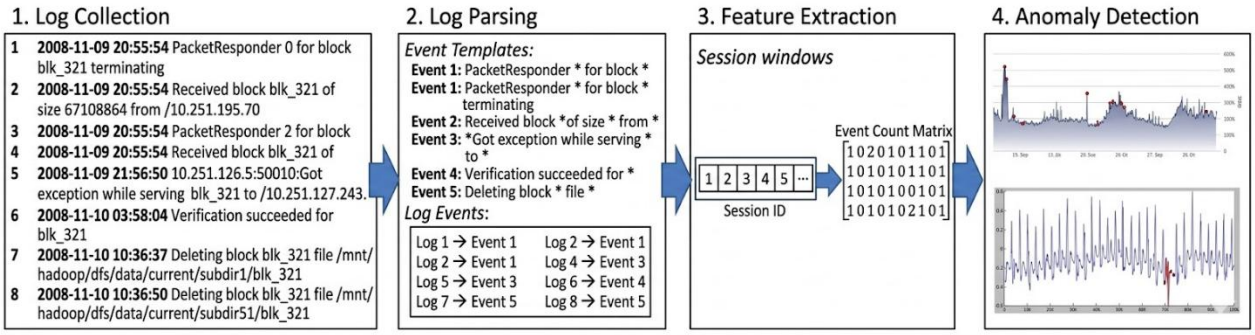
Trong số các mô hình trên, báo cáo lựa chọn Decision Tree làm mô hình đề xuất chính. Lý do là mô hình này vừa cho hiệu quả phát hiện tốt trên dữ liệu HDFS, vừa có tính diễn giải cao nhờ cấu trúc cây quyết định rõ ràng. Điều này giúp thuận lợi hơn trong việc phân tích nguyên nhân khiến một phiên log bị gán nhãn bất thường. Ngoài ra, Decision Tree cũng dễ mô tả và triển khai trong quy trình thực nghiệm của báo cáo.

Bên cạnh đó, Logistic Regression và SVM được sử dụng làm các mô hình đối sánh trong nhóm có giám sát, còn Invariant Mining được dùng làm mô hình tham chiếu trong nhóm không giám sát. Việc đối chiếu này giúp đánh giá rõ hơn mức độ phù hợp của mô hình Decision Tree đối với bộ dữ liệu HDFS trong phạm vi nghiên cứu của đề tài.

2.2.2 Kiến trúc tổng thể của mô hình

Trên cơ sở đặc điểm của bộ dữ liệu HDFS và yêu cầu của bài toán phát hiện bất thường từ nhật ký hệ thống, mô hình được xây dựng theo hướng xử lý log theo từng phiên thực thi và thực hiện phân loại mỗi phiên vào một trong hai lớp: bình thường hoặc bất thường. Kiến trúc tổng thể của mô hình bao gồm các khối chức năng chính: thu thập dữ liệu log, phân tích cú pháp log, gom nhóm log theo phiên, trích xuất đặc trưng và huấn luyện, dự đoán bằng mô hình học máy. Cách tổ chức này nhằm bảo đảm dữ liệu đầu vào được xử lý tuần tự, nhất quán và phù hợp với yêu cầu biểu diễn của mô hình phân loại.

Về mặt tổng quát, luồng xử lý của hệ thống có thể được mô tả như sau: dữ liệu log HDFS ban đầu được thu thập từ hệ thống, sau đó trải qua bước phân tích cú pháp nhằm chuyển các thông điệp log từ dạng văn bản bán cấu trúc sang dạng các mẫu sự kiện có cấu trúc. Tiếp theo, các sự kiện này được gom nhóm theo session window dựa trên trường block_id, từ đó hình thành các chuỗi log đại diện cho từng phiên thực thi. Trên cơ sở các chuỗi log này, hệ thống tiến hành trích xuất đặc trưng dưới dạng event count vector. Cuối cùng, các vector đặc trưng được đưa vào mô hình Decision Tree để thực hiện phân loại và đưa ra kết quả dự đoán bất thường. Hình minh họa rõ hơn pipeline xử lý từ log thô đến đầu ra phát hiện bất thường.



Hình 2 - Quy trình xử lý log từ thu thập dữ liệu đến phát hiện bất thường

Trong kiến trúc này, khối thu thập log có nhiệm vụ tiếp nhận dữ liệu nhật ký từ hệ thống HDFS. Đây là dữ liệu đầu vào ban đầu, bao gồm các dòng log chứa thời gian phát sinh, nội dung thông điệp và các tham số động liên quan đến quá trình vận hành. Tuy nhiên, ở trạng thái ban đầu, log chưa thể được đưa trực tiếp vào mô hình học máy do có tính bán cấu trúc và chứa nhiều thành phần biến đổi theo từng lần thực thi.

Tiếp theo, tại khối phân tích cú pháp log, các dòng log được chuẩn hóa thành các mẫu sự kiện. Ở bước này, phần nội dung cố định của thông điệp được giữ lại nhằm phản ánh bản chất của sự kiện, trong khi các tham số động như địa chỉ IP, kích thước block, mã tiến trình hoặc các giá trị biến đổi khác được loại bỏ hoặc thay thế bằng ký hiệu tổng quát. Nhờ đó, nhiều dòng log có cùng ý nghĩa vận hành nhưng khác nhau về giá trị cụ thể có thể được ánh xạ về cùng một mẫu sự kiện, làm giảm mức độ phân mảnh của dữ liệu và tạo điều kiện thuận lợi cho bước biểu diễn đặc trưng tiếp theo.

Sau khi log được chuẩn hóa, hệ thống tiến hành gom nhóm theo session window. Đối với bộ dữ liệu HDFS, việc sử dụng block_id để xác định phiên là phù hợp vì mỗi block_id đại diện cho một tiến trình thao tác trên một khối dữ liệu nhất định. Do đó, các dòng log có cùng block_id được xem là thuộc cùng một phiên thực thi. Cách tổ chức này cho phép duy trì ngữ cảnh vận hành của hệ thống, đồng thời phản ánh được mối liên hệ giữa các sự kiện phát sinh trong cùng một quá trình xử lý dữ liệu.

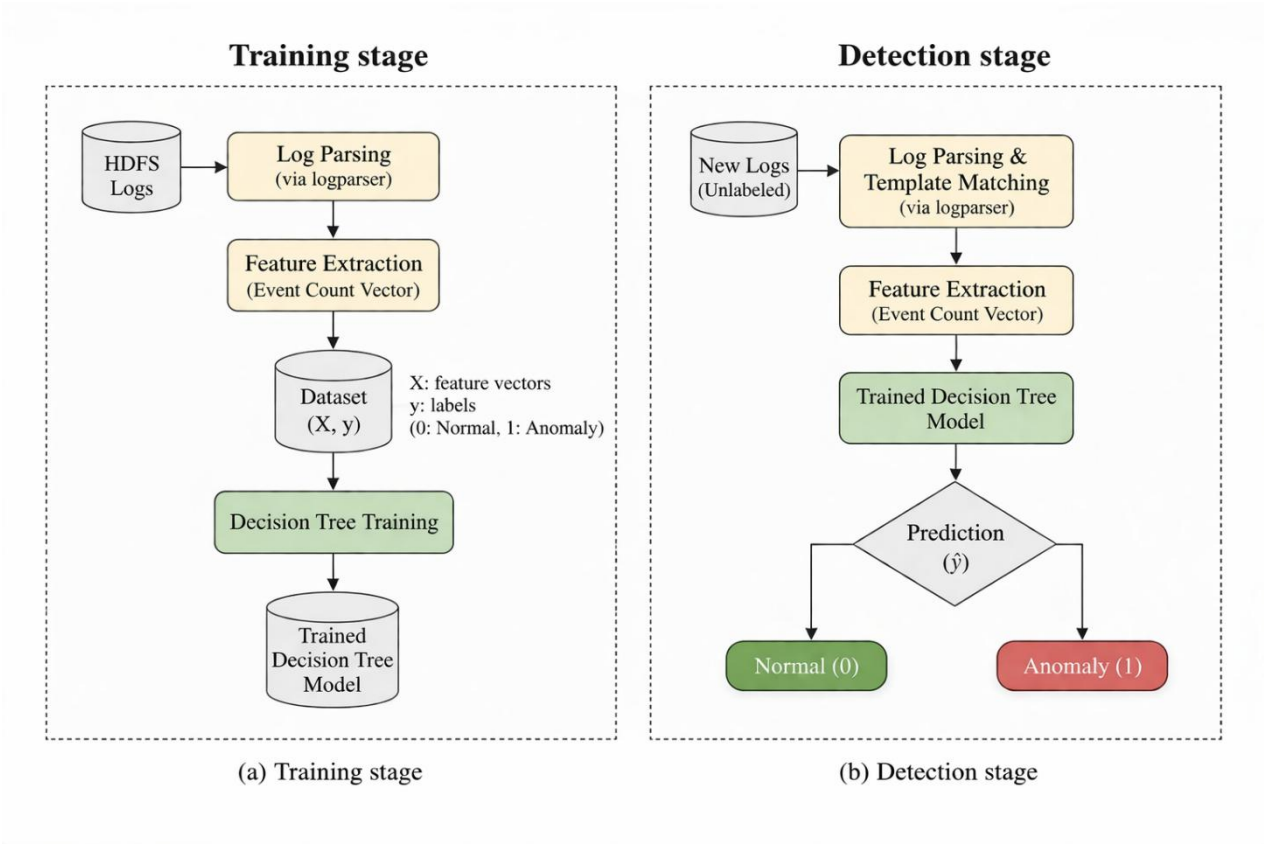
Trên cơ sở các phiên log đã được hình thành, hệ thống thực hiện trích xuất đặc trưng bằng cách đếm số lần xuất hiện của từng loại sự kiện trong mỗi phiên. Mục đích chính của bước này là trích xuất các đặc trưng có giá trị từ các sự kiện log để đưa vào các mô hình phát hiện bất thường. Đầu vào của bước trích xuất đặc trưng là các sự kiện log thu được sau bước phân tích cú pháp log, còn đầu ra là event count matrix (ma trận số đếm sự kiện). Trong nghiên cứu này, dữ liệu được chia theo session window.

Session window dựa trên định danh thay vì dấu thời gian. Định danh được dùng để đánh dấu các đường thực thi khác nhau trong một số dữ liệu log. Chẳng hạn, log HDFS với block_id ghi lại việc cấp phát, ghi, sao chép và xóa một khối nhất định. Vì vậy, ta có thể nhóm log theo các định danh, trong đó mỗi session window có một định danh duy nhất.

Sau khi xây dựng các chuỗi log bằng các kỹ thuật cửa sổ, một ma trận số đếm sự kiện X được tạo ra. Trong mỗi chuỗi log, chúng tôi đếm số lần xuất hiện của mỗi sự kiện log để tạo thành event count vector. Ví dụ, nếu event count vector là $[0, 0, 2, 3, 0, 1, 0]$, điều đó có

nghĩa là sự kiện 3 xuất hiện hai lần và sự kiện 4 xuất hiện ba lần trong chuỗi log đó. Cuối cùng, rất nhiều event count vector được xây dựng để tạo thành event count matrix X, trong đó phần tử $X(i,j)$ ghi lại số lần sự kiện j xuất hiện trong chuỗi log thứ i. Dạng biểu diễn này có ưu điểm là đơn giản, dễ triển khai và phản ánh tương đối tốt sự khác biệt giữa các phiên vận hành bình thường và bất thường trên bộ dữ liệu HDFS.

Sau bước trích xuất đặc trưng, hệ thống chuyển sang giai đoạn huấn luyện và dự đoán bằng mô hình Decision Tree. Đây là mô hình được lựa chọn làm phương pháp chính trong báo cáo do có khả năng phân loại tốt trên dữ liệu biểu diễn theo dạng đếm sự kiện, đồng thời có ưu điểm về tính trực quan và khả năng giải thích kết quả. Trong giai đoạn huấn luyện, mô hình học các quy tắc phân tách giữa hai lớp dữ liệu từ tập mẫu đã được gán nhãn. Trong giai đoạn phát hiện, các phiên log mới được xử lý theo cùng pipeline, sau đó đưa vào mô hình đã huấn luyện để dự đoán nhãn đầu ra là bình thường (0) hoặc bất thường (1). Hình 3 thể hiện rõ hơn mối quan hệ giữa giai đoạn huấn luyện và giai đoạn phát hiện.



Hình 3 - Kiến trúc tổng thể của mô hình trong giai đoạn huấn luyện và giai đoạn phát hiện

Từ góc độ kiến trúc hệ thống, có thể thấy mô hình được tổ chức thành hai giai đoạn rõ ràng. Ở giai đoạn huấn luyện, dữ liệu log HDFS được phân tích cú pháp, trích xuất đặc trưng và kết hợp với nhãn để tạo thành tập dữ liệu huấn luyện (X, y), trong đó X là tập vector đặc trưng và y là tập nhãn tương ứng. Trên cơ sở đó, mô hình Decision Tree được xây dựng để học ra quy luật phân biệt giữa các phiên log bình thường và bất thường. Ở giai đoạn phát hiện, các log mới tiếp tục được xử lý qua cùng pipeline tiền xử lý và trích xuất đặc trưng, sau đó được đưa vào mô hình đã huấn luyện để sinh ra kết quả dự đoán. Cách tổ chức này

bảo đảm sự thống nhất giữa dữ liệu trong pha huấn luyện và dữ liệu trong pha suy luận, qua đó góp phần nâng cao tính ổn định của hệ thống khi triển khai thực tế.

Nhìn chung, kiến trúc tổng thể được đề xuất bảo đảm hai yêu cầu quan trọng của bài toán. Thứ nhất, hệ thống chuyển đổi log từ dạng văn bản bán cấu trúc sang dạng số có thể xử lý được bằng các thuật toán học máy. Thứ hai, quá trình tổ chức dữ liệu theo session window giúp bảo toàn ngữ cảnh thực thi của hệ thống, đặc biệt là mối liên hệ giữa các sự kiện cùng thuộc một block dữ liệu trong HDFS. Nhờ đó, mô hình không chỉ đạt hiệu quả phát hiện cao mà còn phù hợp với mục tiêu nghiên cứu, trình bày và triển khai trong phạm vi báo cáo này.

2.3 Kết chương

Chương 2 đã trình bày mô hình phát hiện bất thường từ nhật ký hệ thống được đề xuất. Trên cơ sở phát biểu bài toán và xác định rõ đầu vào, đầu ra, chương đã xây dựng kiến trúc tổng thể gồm các bước: phân tích cú pháp log, gom nhóm theo session window, trích xuất đặc trưng và huấn luyện mô hình phát hiện bất thường. Đồng thời, chương cũng mô tả quy trình huấn luyện, kiểm thử và thực hiện tổng quát nhằm bảo đảm tính logic và khả năng triển khai của mô hình trên dữ liệu thực tế.

CHƯƠNG 3. THỰC NGHIỆM VÀ ĐÁNH GIÁ

Chương 3 trình bày quá trình thực nghiệm và đánh giá mô hình phát hiện bất thường từ nhật ký hệ thống trên bộ dữ liệu HDFS. Nội dung chương tập trung mô tả mục tiêu thực nghiệm, bộ dữ liệu sử dụng, các độ đo đánh giá và kết quả thu được của các mô hình có giám sát và không giám sát. Trên cơ sở đó, chương đưa ra các nhận xét về hiệu năng phát hiện, thời gian xử lý, khả năng mở rộng và lựa chọn mô hình phù hợp cho bài toán nghiên cứu.

3.1 Mục tiêu thực nghiệm

Chương này trình bày quá trình thực nghiệm và đánh giá mô hình phát hiện bất thường trên bộ dữ liệu nhật ký HDFS. Mục tiêu chính của thực nghiệm là kiểm tra mức độ hiệu quả của các mô hình học máy trong việc phân biệt giữa phiên làm việc bình thường và phiên làm việc bất thường, đồng thời làm rõ tính phù hợp của mô hình Decision Tree đã được lựa chọn ở Chương 2.

Bên cạnh đó, chương này cũng xem xét hiệu quả của một số phương pháp đối sánh để làm cơ sở so sánh, bao gồm các mô hình có giám sát như Logistic Regression, Decision Tree, SVM và các mô hình không giám sát như Log Clustering, Invariant Mining và PCA. Việc so sánh này giúp làm rõ vị trí của mô hình đề xuất trong toàn bộ bài toán phát hiện bất thường từ nhật ký hệ thống.

3.2 Bộ dữ liệu và thiết lập thực nghiệm

Dữ liệu HDFS được thu thập từ nền tảng Amazon EC2, có thời gian ghi nhận 38.7 giờ, dung lượng khoảng 1.55GB, chứa tổng cộng 11.175.629 thông điệp log. Log HDFS ghi lại một block_id duy nhất cho mỗi thao tác trên khối như cấp phát, ghi, sao chép và xóa. Vì vậy, các thao tác trong log có thể được nắm bắt một cách tự nhiên hơn bằng session window, bởi mỗi block_id duy nhất có thể được dùng để cắt log thành một tập chuỗi log. Sau đó, tệp log này đã được trích xuất đặc trưng từ các chuỗi log này và tạo ra 575.061 event count vector. Trong số đó, 16.838 mẫu được đánh dấu là bất thường.

Đối với các mô hình học có giám sát, dữ liệu được chia theo tỷ lệ 80% cho huấn luyện và 20% cho kiểm thử. Cách chia này bảo đảm rằng các sự kiện xảy ra trước được dùng để xây dựng mô hình, còn các sự kiện xuất hiện sau được dùng để đánh giá khả năng dự đoán của mô hình trên dữ liệu mới. Các chỉ số đánh giá được sử dụng gồm Precision, Recall và F1-score, trong đó Precision phản ánh độ chính xác của các cảnh báo bất thường, Recall phản ánh khả năng phát hiện đầy đủ các bất thường thực sự, còn F1-score là trung bình điều hòa giữa hai chỉ số trên.

HDFS là bộ dữ liệu rất đáng chú ý vì nó có những đặc điểm làm cho bài toán phát hiện bất thường trở nên sạch và có cấu trúc hơn so với nhiều bộ log khác. Thứ nhất, mỗi thao tác liên quan đến block đều có block_id, tạo điều kiện tự nhiên để gom nhóm log theo session. Thứ hai, HDFS chỉ có 29 loại sự kiện, ít hơn nhiều so với BGL là 385 loại sự kiện. Điều đó có nghĩa không gian đặc trưng của HDFS gọn hơn, dễ học hơn và ít nhiễu hơn. Thứ ba, khi

log được gom theo block_id, mức tương quan giữa các sự kiện trong một chuỗi log tăng lên đáng kể. Một chuỗi log HDFS vì vậy phản ánh khá rõ một đường thực thi liên quan đến block, giúp các mô hình có giám sát và một số mô hình không giám sát đạt kết quả tốt hơn.

3.3 Các độ đo đánh giá

Để đánh giá hiệu quả của mô hình, báo cáo sử dụng các chỉ số Precision, Recall và F1-score. Đây là các độ đo phổ biến trong bài toán phân loại nhị phân và đặc biệt phù hợp với bối cảnh dữ liệu mất cân bằng như phát hiện bất thường.

Precision phản ánh tỷ lệ các phiên được dự đoán là bất thường mà thực sự bất thường. Chỉ số này cho biết mức độ chính xác của các cảnh báo mà mô hình tạo ra. Precision được xác định bởi:

$$Precision = \frac{TP}{TP + FP}$$

Recall phản ánh tỷ lệ các phiên bất thường thực sự được mô hình phát hiện. Đây là chỉ số quan trọng trong các hệ thống giám sát, vì bỏ sót bất thường có thể dẫn đến hậu quả nghiêm trọng. Recall được xác định bởi:

$$Recall = \frac{TP}{TP + FN}$$

F1-score là trung bình điều hòa giữa Precision và Recall, dùng để đánh giá tổng hợp hiệu quả của mô hình. F1-score được xác định bởi:

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

Trong đó:

- TP là số mẫu bất thường được phát hiện đúng.
- FP là số mẫu bình thường bị dự đoán nhầm là bất thường.
- FN là số mẫu bất thường bị bỏ sót.
- TN là số mẫu bình thường được dự đoán đúng.

Ba độ đo này cho phép đánh giá mô hình ở cả hai khía cạnh: hạn chế cảnh báo giả và hạn chế bỏ sót bất thường. Đây là cơ sở phù hợp để so sánh giữa các phương pháp phát hiện khác nhau trên cùng bộ dữ liệu HDFS.

3.4 Kết quả thực nghiệm trên HDFS

3.4.1 Kết quả của các mô hình có giám sát

Trong nhóm các mô hình có giám sát, báo cáo tiến hành thực nghiệm trên ba thuật toán gồm Logistic Regression (LR), Support Vector Machine (SVM) và Decision Tree nhằm đánh giá khả năng phân loại giữa các phiên log bình thường và bất thường trên bộ dữ liệu HDFS. Các mô hình này được huấn luyện trên tập dữ liệu huấn luyện sau khi đã tiền xử lý

và biểu diễn dưới dạng vector đặc trưng, sau đó được đánh giá trên tập kiểm thử thông qua các độ đo Precision, Recall và F1-score.

Kết quả thực nghiệm của nhóm mô hình có giám sát được trình bày trong Bảng 1 và Bảng 2. Trong hai bảng này, mỗi mô hình được đánh giá trên cả tập huấn luyện và tập kiểm thử nhằm phản ánh không chỉ hiệu quả phát hiện mà còn mức độ tổng quát hóa của mô hình khi áp dụng trên dữ liệu chưa quan sát trước đó.

Bảng 1. Kết quả thực nghiệm của các mô hình có giám sát trên tập huấn luyện

Mô hình	**Precision (%)**	**Recall (%)**	**F1-score (%)**
LR	86.43	71.05	77.99
SVM	96.07	97.18	96.62
Decision Tree	99.84	99.96	99.90

Bảng 2. Kết quả thực nghiệm của các mô hình có giám sát trên tập kiểm thử

Mô hình	**Precision (%)**	**Recall (%)**	**F1-score (%)**
LR	86.36	72.00	78.53
SVM	96.01	97.12	96.56
Decision Tree	99.79	99.88	99.84

Từ kết quả thu được, có thể nhận thấy mô hình Decision Tree đạt hiệu năng tốt nhất trong nhóm các phương pháp có giám sát. Cụ thể, trên tập huấn luyện, mô hình này đạt Precision = 99.84%, Recall = 99.96% và F1-score = 99.90%. Trên tập kiểm thử, mô hình đạt Precision = 99.79%, Recall = 99.88% và F1-score = 99.84%. Kết quả này cho thấy mô hình Decision Tree không chỉ đạt độ chính xác rất cao mà còn có khả năng tổng quát hóa tốt khi chênh lệch giữa tập huấn luyện và tập kiểm thử là rất nhỏ.

Đứng thứ hai trong nhóm mô hình có giám sát là SVM, với kết quả trên tập kiểm thử đạt Precision = 96.01%, Recall = 97.12% và F1-score = 96.56%. Trong khi đó, Logistic Regression cho kết quả thấp hơn với Precision = 86.36%, Recall = 72% và F1-score = 78.53% trên tập kiểm thử. Điều này cho thấy Logistic Regression chưa khai thác tốt ranh giới phân loại giữa log bình thường và log bất thường trong bộ dữ liệu HDFS so với hai mô hình còn lại.

Từ các kết quả trên, có thể thấy rằng nhóm mô hình có giám sát nhìn chung đạt độ chính xác cao do tận dụng được thông tin nhãn trong quá trình huấn luyện. Trong đó, Decision Tree là mô hình nổi bật nhất nhờ đạt sự cân bằng rất tốt giữa Precision và Recall, qua đó nâng cao hiệu quả phát hiện bất thường mà không làm gia tăng đáng kể số lượng cảnh báo sai. Kết quả này cho thấy các phương pháp có giám sát là lựa chọn phù hợp khi bộ dữ liệu huấn luyện có sẵn nhãn đầy đủ và đáng tin cậy.

3.4.2 Kết quả của các mô hình không giám sát

Bên cạnh nhóm mô hình có giám sát, báo cáo tiếp tục thực nghiệm với các phương pháp không giám sát hoặc bán giám sát, bao gồm PCA, InvariantsMiner, LogClustering và Isolation Forest nhằm đánh giá khả năng phát hiện bất thường trong điều kiện không phụ thuộc nhiều vào nhãn dữ liệu. Đây là hướng tiếp cận có ý nghĩa thực tiễn cao trong bối cảnh dữ liệu log thực tế thường rất lớn và việc gán nhãn thủ công là tốn thời gian, công sức.

Kết quả thực nghiệm của các mô hình không giám sát được trình bày trong Bảng 3 và Bảng 4. Các mô hình được đánh giá dựa trên các độ đo Precision, Recall và F1-score trên tập kiểm thử để đảm bảo khả năng so sánh nhất quán với nhóm mô hình có giám sát.

Bảng 3. Kết quả thực nghiệm của các mô hình không giám sát trên tập huấn luyện

Mô hình	Precision (%)	Recall (%)	F1-score (%)
PCA	97.50	63.46	76.88
InvariantsMiner	89.44	100	94.42
LogClustering	100	71.96	83.69
Isolation Forest	79.22	79.02	79.12

Bảng 4. Kết quả thực nghiệm của các mô hình không giám sát trên tập kiểm thử

Mô hình	Precision (%)	Recall (%)	F1-score (%)
PCA	97.35	64.40	77.52
InvariantsMiner	89.94	100	94.14
LogClustering	100	72.03	83.74
Isolation Forest	78.19	79.51	78.85

Kết quả cho thấy mô hình InvariantsMiner đạt hiệu năng tốt nhất trong nhóm không giám sát với Precision = 89.44%, Recall = 100% và F1-score = 94.42% trên tập huấn luyện và Precision = 89.94%, Recall = 100% và F1-score = 94.14% trên tập kiểm thử. Trong khi đó, mô hình PCA có kết quả thấp hơn, đặc biệt ở chỉ số Recall khoảng 64%, cho thấy phương pháp này còn gặp khó khăn trong việc nhận diện đầy đủ các phiên log bất thường.

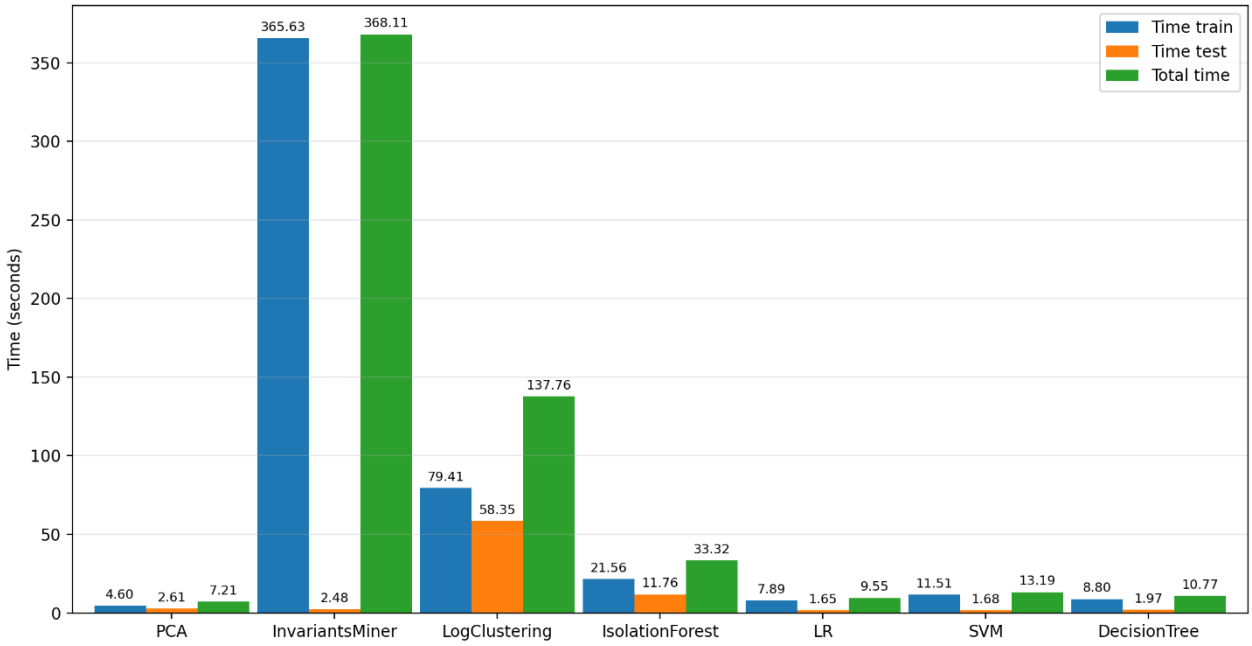
So với nhóm mô hình có giám sát, hiệu năng của các phương pháp không giám sát thường thấp hơn do không tận dụng trực tiếp được thông tin nhãn trong giai đoạn huấn luyện. Tuy nhiên, ưu điểm của nhóm này là có khả năng áp dụng linh hoạt trong các hệ thống thực tế, nơi dữ liệu bất thường hiếm và khó gán nhãn. Vì vậy, mặc dù kết quả có thể chưa đạt mức cao như các mô hình có giám sát, nhóm phương pháp này vẫn có giá trị lớn trong các kịch bản triển khai thực tế.

3.4.3 So sánh thời gian xử lý của các mô hình

Ngoài độ chính xác phát hiện, thời gian xử lý cũng là một tiêu chí quan trọng khi đánh giá mô hình phát hiện bất thường từ log. Trong môi trường giám sát hệ thống, một mô hình hiệu quả không chỉ cần phát hiện chính xác mà còn cần có thời gian huấn luyện và kiểm thử

đủ ngắn để có thể triển khai trên dữ liệu quy mô lớn hoặc trong các hệ thống có yêu cầu phản hồi nhanh.

Để đánh giá khía cạnh này, báo cáo tiến hành đo thời gian huấn luyện, thời gian kiểm thử và tổng thời gian xử lý của các mô hình trên cùng một bộ dữ liệu và cùng cấu hình thực nghiệm. Kết quả được trực quan hóa trong Hình 4.



Hình 4 – So sánh thời gian xử lý của các mô hình

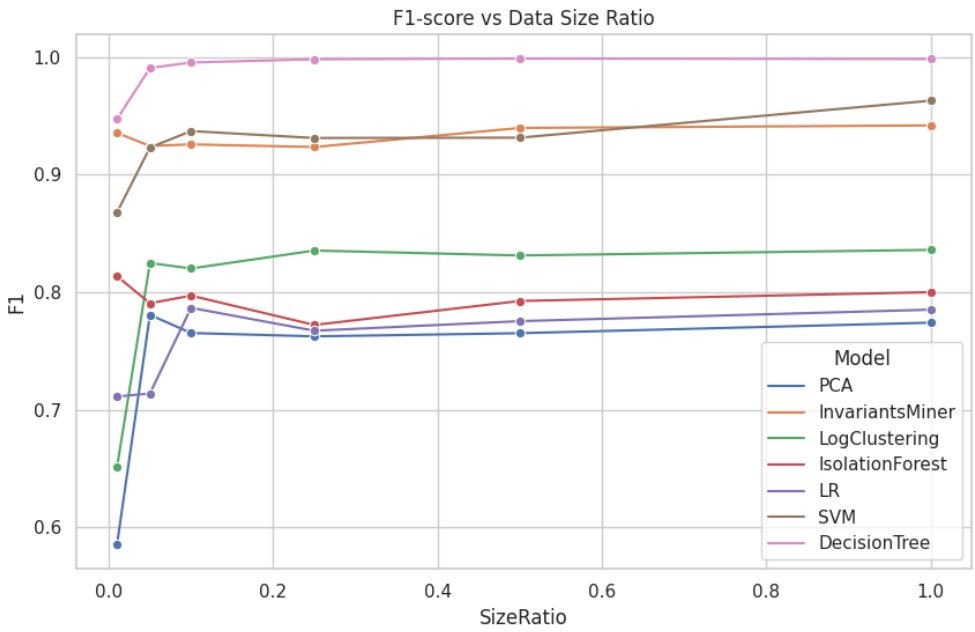
Từ các biểu đồ có thể nhận thấy mô hình PCA có thời gian huấn luyện thấp nhất, đạt khoảng 4.60 giây, trong khi InvariantsMiner là mô hình có thời gian huấn luyện lớn nhất với 365.63 giây. Đối với thời gian kiểm thử, LR cho kết quả nhanh nhất với 1.66 giây, trong khi LogClustering cần tới 58.35 giây để hoàn thành quá trình dự đoán trên tập kiểm thử. Xét về tổng thời gian xử lý, PCA là mô hình có chi phí tính toán thấp nhất với 7.21 giây, còn InvariantsMiner là phương pháp tốn thời gian nhất với 368.11 giây.

Từ kết quả này có thể thấy rằng không phải mô hình có hiệu năng phát hiện cao nhất cũng là mô hình có tốc độ xử lý tốt nhất. Một số phương pháp tuy cho giá trị F1-score cao nhưng lại yêu cầu thời gian huấn luyện hoặc suy luận lớn, gây hạn chế khi áp dụng trong môi trường thực tế. Ngược lại, một số mô hình có thời gian xử lý thấp hơn nhưng hiệu năng phát hiện chỉ ở mức trung bình. Do đó, việc lựa chọn mô hình phù hợp cần dựa trên sự cân đối giữa chất lượng phát hiện và chi phí tính toán.

3.4.4 Đánh giá khả năng mở rộng của các mô hình theo quy mô dữ liệu

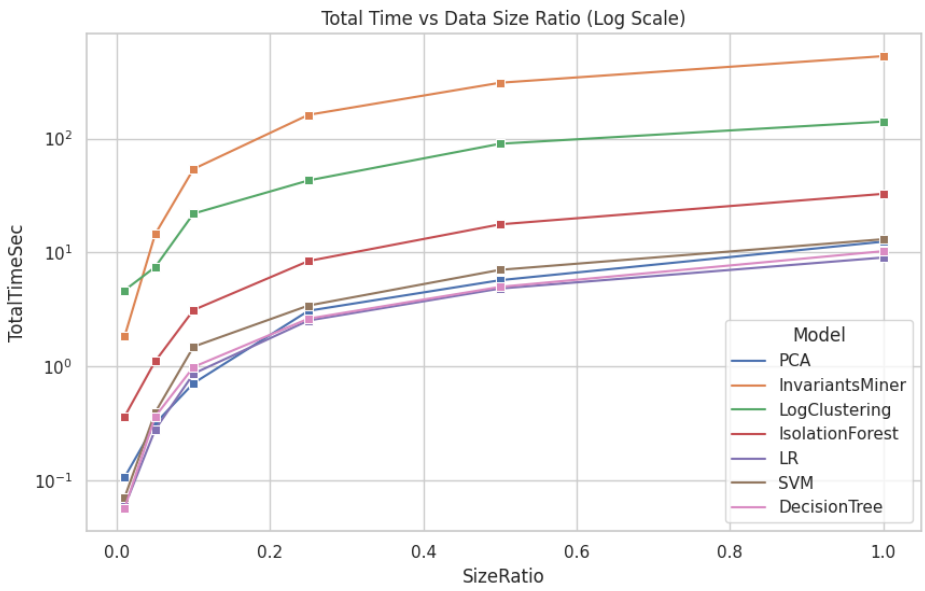
Để đánh giá khả năng mở rộng của các mô hình, báo cáo khảo sát sự thay đổi của hiệu năng và thời gian xử lý khi quy mô dữ liệu đầu vào tăng dần theo các mức 1%, 5%, 10%, 25%, 50% và 100% của toàn bộ tập dữ liệu. Ở mỗi mốc, các mô hình được huấn luyện và kiểm thử lại để ghi nhận chất lượng phát hiện cũng như chi phí tính toán tương ứng.

Kết quả về F1-score trên tập kiểm thử được thể hiện trong Hình 5. Từ biểu đồ có thể thấy rằng khi quy mô dữ liệu tăng lên, phần lớn các mô hình đều có xu hướng cải thiện hoặc duy trì hiệu năng ổn định. Trong đó, Decision Tree là mô hình nổi bật nhất khi F1-score tăng từ 0,947 ở mức dữ liệu 1% lên 0,998 ở mức 100%, cho thấy khả năng tận dụng dữ liệu bổ sung rất hiệu quả. SVM cũng đạt kết quả khả quan khi tăng từ 0,868 lên 0,963. Đối với nhóm không giám sát, InvariantsMiner duy trì hiệu năng cao và ổn định, dao động quanh mức 0,924–0,942, trong khi PCA và Isolation Forest có mức cải thiện thấp hơn. Kết quả này cho thấy các mô hình có giám sát, đặc biệt là Decision Tree và SVM, có khả năng thích nghi tốt hơn khi dữ liệu ngày càng mở rộng.



Hình 5 – F1-score của các mô hình theo quy mô dữ liệu

Bên cạnh hiệu năng phát hiện, Hình 6 trình bày tổng thời gian xử lý của các mô hình theo từng mốc dữ liệu. Có thể nhận thấy rằng thời gian xử lý của tất cả các mô hình đều tăng khi quy mô dữ liệu lớn hơn, tuy nhiên tốc độ tăng là khác nhau. InvariantsMiner là mô hình có chi phí tính toán tăng mạnh nhất, từ 1,831 giây lên 528,255 giây, trong khi LogClustering cũng tăng đáng kể từ 4,652 giây lên 140,750 giây. Ngược lại, Decision Tree, LR và SVM có mức tăng ổn định hơn và vẫn giữ được thời gian xử lý ở mức chấp nhận được tại mốc dữ liệu lớn nhất. Đáng chú ý, Decision Tree vừa đạt F1-score cao, vừa có tổng thời gian xử lý tương đối thấp hơn nhiều so với các mô hình như InvariantsMiner hay LogClustering.



Hình 6 - Tổng thời gian xử lý của các mô hình theo quy mô dữ liệu

Từ các kết quả trên có thể thấy rằng khả năng mở rộng là yếu tố quan trọng khi lựa chọn mô hình cho bài toán phát hiện bất thường từ log. Một mô hình phù hợp không chỉ cần duy trì độ chính xác cao khi dữ liệu tăng mà còn phải có chi phí tính toán hợp lý. Trong số các phương pháp được khảo sát, Decision Tree cho thấy sự cân bằng tốt nhất giữa hiệu năng phát hiện và khả năng mở rộng, do đó là lựa chọn phù hợp cho các kịch bản triển khai thực tế với dữ liệu log quy mô lớn.

3.5 Đánh giá và nhận xét

Từ các kết quả thực nghiệm có thể thấy rằng các mô hình có giám sát cho hiệu quả phát hiện bất thường tốt hơn so với nhóm mô hình không giám sát. Trong nhóm có giám sát, Decision Tree là mô hình đạt kết quả cao nhất trên tập kiểm thử, tiếp theo là SVM, trong khi Logistic Regression cho kết quả thấp hơn. Điều này cho thấy với dữ liệu log đã có nhãn, các mô hình phân loại có khả năng học ranh giới quyết định tốt sẽ cho hiệu năng cao hơn.

Đối với nhóm mô hình không giám sát, Invariants Miner là phương pháp đạt kết quả nổi bật nhất. Các mô hình còn lại như Log Clustering, Isolation Forest và PCA cho hiệu năng thấp hơn, đặc biệt ở khả năng cân bằng giữa Precision và Recall. Tuy vậy, nhóm mô hình này vẫn có ý nghĩa thực tiễn trong các trường hợp dữ liệu thiếu nhãn hoặc khó gán nhãn.

Về thời gian xử lý, kết quả cho thấy có sự khác biệt rõ giữa các mô hình. Một số phương pháp như PCA, LR và Decision Tree có thời gian xử lý thấp hơn, trong khi Invariants Miner và Log Clustering tốn nhiều thời gian hơn đáng kể. Trong số các mô hình đã khảo sát, Decision Tree là phương pháp đạt sự cân bằng tốt nhất giữa độ chính xác và chi phí tính toán.

Khi đánh giá theo quy mô dữ liệu, hầu hết các mô hình đều cải thiện hoặc duy trì hiệu năng khi dữ liệu tăng lên. Trong đó, Decision Tree tiếp tục cho thấy ưu thế rõ rệt khi vừa

duy trì F1-score cao, vừa có thời gian xử lý tăng theo xu hướng ổn định. Kết quả này cho thấy mô hình phù hợp với bài toán phát hiện bất thường trên dữ liệu log có quy mô lớn.

3.6 Kết chương

Chương 3 đã trình bày quá trình thực nghiệm và đánh giá mô hình phát hiện bất thường trên bộ dữ liệu HDFS. Kết quả cho thấy các mô hình có giám sát đạt hiệu năng phát hiện cao hơn so với các mô hình không giám sát khi dữ liệu đã được tổ chức theo session window và biểu diễn dưới dạng vector đặc trưng. Trong số các mô hình được khảo sát, Decision Tree cho thấy sự cân bằng tốt giữa độ chính xác, khả năng diễn giải và chi phí tính toán. Bên cạnh đó, các kết quả đánh giá về thời gian xử lý và khả năng mở rộng cũng cho thấy mô hình này phù hợp với các kịch bản dữ liệu log quy mô lớn. Những kết quả đạt được là cơ sở thực nghiệm quan trọng để khẳng định tính khả thi và hiệu quả của hướng tiếp cận đã đề xuất trong báo cáo.

KẾT LUẬN

Các kết quả đạt được

Nhóm thực hiện đề tài "Phát hiện bất thường từ nhật ký hệ thống bằng các phương pháp học máy" đã hoàn thành việc nghiên cứu, xây dựng quy trình xử lý dữ liệu log và thực nghiệm đánh giá các mô hình phát hiện bất thường trên bộ dữ liệu HDFS. Đề tài đã thực hiện đầy đủ các nội dung đã đăng ký theo đề cương, cụ thể như sau:

- Nghiên cứu tổng quan về bài toán phát hiện bất thường từ nhật ký hệ thống, đặc điểm dữ liệu log và vai trò của bài toán trong giám sát, vận hành hệ thống.
- Nghiên cứu quy trình tiền xử lý dữ liệu log, bao gồm tổ chức log theo session window, trích xuất đặc trưng từ chuỗi sự kiện và biểu diễn dữ liệu dưới dạng vector đặc trưng phục vụ huấn luyện mô hình.
- Tìm hiểu và triển khai thực nghiệm nhiều mô hình học máy cho bài toán phát hiện bất thường, gồm cả nhóm mô hình có giám sát và không giám sát như Logistic Regression, SVM, Decision Tree, PCA, Invariants Miner, Log Clustering và Isolation Forest.
- Tiến hành huấn luyện, kiểm thử và đánh giá các mô hình trên bộ dữ liệu HDFS thông qua các độ đo Precision, Recall, F1-score, đồng thời khảo sát thêm thời gian huấn luyện, thời gian kiểm thử và tổng thời gian xử lý.
- Thực hiện đánh giá khả năng mở rộng của các mô hình theo nhiều mức quy mô dữ liệu khác nhau nhằm phân tích ảnh hưởng của kích thước dữ liệu đến hiệu năng phát hiện và chi phí tính toán.
- Từ các kết quả thực nghiệm, xác định được mô hình phù hợp nhất trong phạm vi nghiên cứu là Decision Tree, do mô hình này đạt độ chính xác cao, thời gian xử lý hợp lý và khả năng mở rộng tốt trên bộ dữ liệu thử nghiệm.

Hướng phát triển

Đề tài này có thể được mở rộng theo các hướng sau:

- Thử nghiệm trên thêm các bộ dữ liệu log khác như BGL hoặc các bộ dữ liệu log thực tế nhằm đánh giá tính tổng quát của mô hình.
- Nghiên cứu các phương pháp biểu diễn dữ liệu log giàu thông tin hơn, không chỉ dựa trên tần suất sự kiện mà còn khai thác thứ tự xuất hiện và khoảng cách thời gian giữa các sự kiện.
- Mở rộng so sánh với các mô hình học sâu chuyên dụng cho log anomaly detection nhằm đánh giá rõ hơn ưu, nhược điểm giữa các phương pháp truyền thống và hiện đại.
- Tối ưu hóa thời gian huấn luyện và suy luận của mô hình trên dữ liệu lớn nhằm nâng cao khả năng triển khai thực tế.

TÀI LIỆU THAM KHẢO

[1] S. He, J. Zhu, P. He, and M. R. Lyu, "Experience Report: System Log Analysis for Anomaly Detection," *2016 IEEE 27th International Symposium on Software Reliability Engineering (ISSRE)*, 2016, pp. 207–218.

[2] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An Online Log Parsing Approach with Fixed Depth Tree," *2017 IEEE International Conference on Web Services (ICWS)*, 2017, pp. 33–40.

[3] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning," *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2017, pp. 1285–1298.

[4] W. Meng, Y. Liu, Y. Zhu, S. Zhang, D. Pei, Y. Liu, Y. Chen, R. Zhang, S. Tao, P. Sun, and R. Zhou, "LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs," *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence (IJCAI)*, 2019, pp. 4739–4745.

[5] J. Zhu, S. He, J. Liu, P. He, Q. Xie, Z. Zheng, and M. R. Lyu, "Tools and Benchmarks for Automated Log Parsing," *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2019, pp. 121–130.

[6] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," *2008 Eighth IEEE International Conference on Data Mining (ICDM)*, 2008, pp. 413–422.

[7] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying Density-Based Local Outliers," *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, 2000, pp. 93–104.

[8] Y. Lee, J. Kim, and P. Kang, "LAnoBERT: System log anomaly detection based on BERT masked language model," *Applied Soft Computing*, vol. 146, 2023, Art. no. 110689.

[9] G. Tian, N. Luktarhan, H. Wu, and Z. Shi, "CLDTLog: System Log Anomaly Detection Method Based on Contrastive Learning and Dual Objective Tasks," *Sensors*, vol. 23, no. 11, 2023, Art. no. 5042.

[10] C. Zhang, T. Jia, G. Shen, P. Zhu, and Y. Li, "MetaLog: Generalizable Cross-System Anomaly Detection from Logs with Meta-Learning," *ICSE 2024*.

[11] S. Zhang et al., "End-to-End AutoML for Unsupervised Log Anomaly Detection," *ASE 2024*.

[12] S. Ali et al., "A comprehensive study of machine learning techniques for log-based anomaly detection," *Empirical Software Engineering*, vol. 30, 2025, Art. no. 129.